



Universidad Don Bosco
FACULTAD DE INGENIERÍA
ESCUELA DE COMPUTACIÓN
Desarrollo de Aplicaciones con Software Propietario
Guía 5: Métodos



I. OBJETIVOS

Que el estudiante sea capaz de:

- Redactar expresiones con funciones matemáticas para resolver cálculos complejos
- Determinar la función de formato a utilizar al momento de almacenar/recibir/mostrar datos dentro del programa.
- Crear su primer procedimiento y función en soluciones C#.NET

II. INTRODUCCION TEORICA

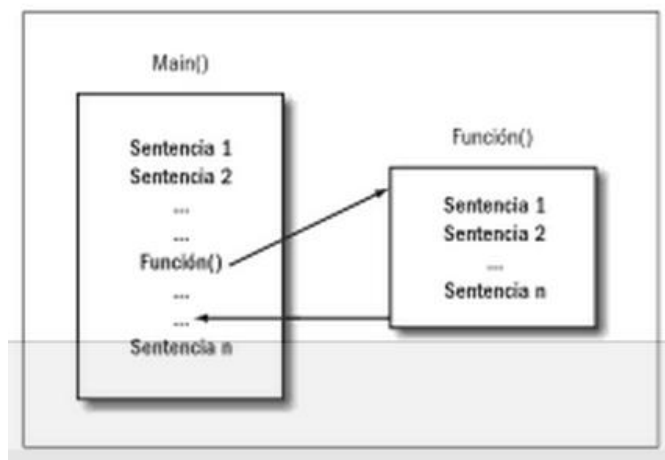
LAS FUNCIONES

Las funciones y métodos son elementos muy importantes de programación, estos serán utilizados frecuentemente en C#, especialmente cuando se avance a la programación orientada a objetos.

Las funciones dan muchas ventajas y permiten desarrollar aplicaciones de forma más rápida y ordenada.

La función es un elemento del programa que contiene código y se puede ejecutar, es decir, llevar a cabo una función. La función puede llamarse o invocarse cuando sea necesario y entonces el código que se encuentra en su interior se ejecutará. Una vez que la función termina de ejecutarse, el programa continúa en la sentencia siguiente de donde fue llamada (Ver la figura 1).

Figura 1.



Se puede observar cómo la ejecución del programa se dirige a la función y luego regresa a la sentencia siguiente de donde fue invocada.

Para que las funciones sean útiles deben estar especializadas, es decir, que una función debe hacer solamente una cosa y hacerla bien. Dentro de la programación orientada a objetos, las clases tienen código y éste se encuentra dentro de funciones llamadas métodos. Por ejemplo, cuando se convierte de cadena a un entero, se hace uso de la función llamada `ToInt32()` que se encuentra dentro de la clase `Convert`.

Es importante seleccionar correctamente el nombre de la función. Éste debe hacer referencia al tipo de trabajo que

lleva a cabo la función.

Uno de los principales beneficios del uso de funciones y procedimientos es la reutilización de código. Los procedimientos creados en un programa se pueden utilizar en otros programas o proyectos, frecuentemente con

poca o nula modificación. Los procedimientos son útiles para tareas repetidas o compartidas, como cálculos utilizados frecuentemente.

Las funciones constan de cinco partes:

```

Modificador tipo nombre (parámetros)
{
    Código
}
    
```

Las funciones pueden regresar información y ésta puede ser cadena, entero, flotante o cualquier otro tipo. En la sección de tipo se tiene que indicar precisamente el tipo de información que regresa. Si la función no retorna ningún valor, entonces se indica que el tipo de la función es void.

Las funciones pueden necesitar datos o información para trabajar, esa información se le da por medio de sus parámetros, que son una lista de variables que reciben estos datos. Si la función no necesita utilizar parámetros, entonces se pueden dejar los paréntesis vacíos. No se debe de olvidar colocar los paréntesis aunque no haya parámetros.

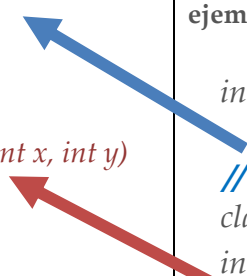
Las funciones pueden tener 4 tipos de parámetros:

- Parámetros de entrada: tienen valores que son enviados a la función pero la función no puede cambiar esos valores.
- Parámetros de salida: no tienen valor cuando son enviados a la función pero la función puede darles un valor y enviar el valor de vuelta al invocador.
- Parámetros por referencia: introducen una referencia en otro valor. Tienen un valor de entrada para la función y ese valor puede ser cambiado dentro de la función.
- Parámetros Params: definen un número variable de argumentos en una lista.

El código de la función se coloca adentro de un bloque de código. En esta sección se puede colocar cualquier código válido de C#, es decir, declaración de variables, ciclos, estructuras selectivas e incluso invocaciones a funciones.

Al ser declarada una función puede llevar un modificador antes del tipo. Los modificadores cambian como trabaja una función. Por ejemplo una función declarada con modificador conocido como static, permite utilizar la función sin necesidad de declarar un objeto de la clase a la que pertenece, ver ejemplo 1.

Ejemplo1. claseOperaciones.cs

Declaración de claseOperaciones	Explicación
<pre> public class claseOperaciones { public int Suma1(int x, int y) { return x + y; } public static int Suma2(int x, int y) { return x + y; } } </pre>	Ambas funciones de claseOperaciones hacen exactamente lo mismo, sin embargo, uno es estática y la otra no. Veamos como usamos cada una de ellas: ejemploUso.aspx.cs <pre> int x = 2, y = 5; // Ejemplo de uso de la función NO estática claseOperaciones opera = new claseOperaciones(); int total1 = opera.Sum1(x, y); // Ejemplo de uso de la función estática int total2 = claseOperaciones.Sum2(x, y); </pre> 

¿Cuándo usar funciones estáticas y en qué caso? ,

Pues eso depende de la funcionalidad que se le vaya a dar.

En el ejemplo que se ha presentado anteriormente que tiene como modificador static, ya que no aporta nada si inicializa la clase. Simplemente quiero usar una función que está almacenada en la claseOperaciones.

Nota:

Las funciones no pueden ser declaradas dentro de otra función. Esto lleva a un error de sintaxis y de lógica que debe ser corregido lo antes posible.

Se tienen cuatro tipos de funciones: las que sólo ejecutan código, las que reciben parámetros, las que regresan valores y las que reciben parámetros y reciben valores.

FUNCIONES INTRÍNSECAS

Las funciones intrínsecas son proporcionadas directamente por el lenguaje de programación. Las funciones más comunes son las de conversión de tipos de datos y matemáticas, que ya se han visto en guías anteriores; las funciones de fecha se verán en el procedimiento de esta guía.

Funciones de manejo de Fechas

De manera similar a las funciones de manejo de cadenas de caracteres incluidas con la clase String, C#.net ofrece el tipo de dato **Date**, la cual es una clase con funciones dedicadas a datos de fechas y operaciones con las mismas.

Muchas de estas funciones pueden ser invocadas con el método compartido y el resto se invocan con el método de instancia de clase.

Algunas de las funciones/métodos disponibles son las siguientes:

Función	Resultado
Now	Devuelve la fecha y hora actual del sistema, en un formato largo.
Today	Esta función retorna la fecha actual del sistema.
Year	Devuelve el año de una fecha especificada.
Month	Obtiene el Número de mes de una fecha especificada.
Day	Obtiene el número de día de una fecha enviada o especificada
DayOfWeek	Obtiene el número del día de la semana, tomando el domingo como valor número 1.

FUNCIONES CREADAS POR EL USUARIO

El desarrollo de una aplicación, especialmente si se trata de un proyecto de gran tamaño, es más fácil si se divide en piezas más pequeñas. El uso de procedimientos puede ayudarnos a agrupar nuestro código en secciones lógicas y condensar tareas repetidas o compartidas, como cálculos utilizados frecuentemente.

Los procedimientos son las sentencias de código ejecutable de un programa. Las instrucciones de un procedimiento están delimitadas por una instrucción de declaración y una llave de cierre.

En esta guía, aprenderemos a crear y utilizar Subrutinas y Funciones.

Procedimientos y funciones en C#

Todas las instrucciones deben estar incluidas en un procedimiento o función, a las que llamamos mediante un identificador.

A estas funciones y procedimientos podemos pasarles datos por medio de parámetros.

En C# tenemos 4 tipos de procedimientos:

1. Procedimientos que ejecutan un código a petición sin devolver ningún resultado.
2. Funciones que ejecutan un código y devuelven el resultado al código que las llamó.
3. Procedimientos de propiedades que permiten manejar las propiedades de los objetos creados.

4. Procedimientos de operador utilizados para modificar el funcionamiento de un operador cuando se aplica a una clase o una estructura.

Procedimiento

Realizan acciones pero no devuelven un valor al procedimiento que origina la llamada.

Cuando un procedimiento llama a otro procedimiento, se transfiere el control al segundo procedimiento. Luego, al finaliza la ejecución del código del segundo procedimiento, éste devuelve el control al procedimiento que lo invocó.

```
public|private|internal void nombreProcedimiento(|lista de parametros|)
{
    ...
    Código a ejecutar por el procedimiento
    ...
}
```

La visibilidad de un procedimiento se determina por las palabras reservadas: **private**, **public** o **internal**.

Si no se indica la visibilidad, por defecto se establece que es **public**

Los controladores de eventos son ejemplos de procedimientos, que se ejecutan en respuesta a un evento.

Función

Es similar a un procedimiento, pero al finalizar, devuelve un resultado al código que la invoca. La ejecución de **return** provoca la salida de la función.

```
public|private|internal tipoDatoretorno nombreFuncion(|lista de parametros|)
{
    Código a ejecutar por la funcion
    return (valor_a_Retornar);
    ...
}
```

El valor que devuelve una función al programa que origina la llamada se denomina **valor de retorno**.

La instrucción **return** especifica el valor devuelto, y devuelve el control inmediatamente a programa que origina la llamada. La instrucción return provoca la salida inmediata de una función.

Pueden utilizarse cualquier número de instrucciones **return** dentro de una función.

Procedimiento de propiedad

Se utilizan cuando se requiere modificar y/o recuperar un valor (**set** / **get**) desde el interior de un Objeto.

```
public tipoDatodeLaPropiedad nombrePropiedad {

    get {
        ...
        //código que se ejecuta cuando se lee la propiedad
        ...
        return variable;
    }

    set {
        ...
        //código que se ejecuta durante la asignación de una propiedad
        //Existe una variable que se declara implícitamente y que contiene
        //el valor que se debe asignar a la propiedad
        ...
    }
}
```

```
variable = value;
...
}
```

Se puede crear una propiedad de sólo lectura o sólo escritura, eliminando el bloque set y/o get correspondiente. También, puede implementar automáticamente la encapsulación cuando no haya tratamiento alguno de la siguiente manera.

```
public int tasa { get; set; }
```

Procedimiento de operador

Permite redefinir a un operador estándar del lenguaje C# para utilizarlo en tipos personalizados (como clases o estructuras).

En esta práctica veremos a los procedimientos y funciones.

Accesibilidad del procedimiento

Utilizamos un modificador de acceso para definir la accesibilidad de los procedimientos que escribimos (es decir, el permiso para que otro código invoque al procedimiento). Si no especificamos un modificador de acceso, los procedimientos son declarados public de forma predeterminada.

La siguiente tabla muestra las opciones de accesibilidad para declarar un procedimiento dentro de un módulo:

Modificador de acceso	Descripción
public	Ninguna restricción de acceso
protected	Solamente puede obtener acceso al tipo o miembro de la clase o struct, o bien de una clase derivada de dicha clase.
private	Accesible únicamente en el mismo ensamblado que contiene la declaración
internal	Puede obtener acceso al tipo o miembro cualquier código del mismo ensamblado, pero no de un ensamblado distinto.

Algunos ejemplos de uso de estos modificadores de acceso

Procedimiento	Función
El siguiente código crea un procedimiento denominado acercaDe() que utiliza un cuadro de mensaje para mostrar un nombre de producto y un número de versión: <pre>private void acercaDe() { MessageBox.Show("Mi programa V2.0"); }</pre>	El siguiente código crea una función denominada cuadrado que devuelve el cuadrado de un número entero (int): <pre>public int cuadrado(int numero) { int valor;//valor es el numero al cuadrado valor = numero * numero; return valor; }</pre>

¿Cómo declarar parámetros en procedimientos?

Un procedimiento que realiza tareas repetidas o compartidas utiliza distinta información en cada llamada. Entre procedimientos y funciones, se pueden transferir datos por medio de **Argumentos**.

Cuando definimos un procedimiento, describimos los datos y los tipos de datos para los que el procedimiento está diseñado para aceptar cuando se invoca. Los elementos definidos en el procedimiento se denominan **parámetros**.

Desarrollo de Aplicaciones con Software Propietario

Cuando invocamos el procedimiento, sustituimos un valor actual de cada parámetro. Los valores que asignamos en lugar de los parámetros se denominan **argumentos**.

Además, definimos el modo en el que otros procedimientos pueden pasar argumentos al nuevo procedimiento. Podemos escoger pasar argumentos **por referencia** (con uso de palabras **ref** o también **out**) o **por valor**.

Declarando Parámetros

Cada parámetro de un procedimiento se declara del mismo modo en que declaramos una variable, especificando el nombre del parámetro y su tipo de datos. También podemos especificar el mecanismo de paso y si el parámetro es opcional.

La sintaxis para cada parámetro de un procedimiento es como sigue:

{ ref | out } TipodeDato nombrep parametro

O sino, como un arreglo de parámetros variables

[TipodeDato] { param [] } nombrep parametro

En donde:

- ✓ Param []: utilizado para indicar una lista opcional de parámetros. Proporciona una forma de pasar un numero arbitrario de argumentos
- ✓ No es necesario que los nombres de argumentos utilizados cuando se invoca un procedimiento coincidan con los nombres de parámetros utilizados para definir el procedimiento.

Por valor

Se hace una copia del valor de un argumento hacia el parámetro del procedimiento/función que se invoca.

Si se efectúa un cambio al parámetro en el procedimiento, solo tendrá efecto dentro del procedimiento/función y no se altera la variable del argumento.

La transferencia por valor es la forma predeterminada para enviar valores enteros o decimales, booleanos y estructuras definidas por el usuario. Los demás tipos se pasan por referencia.

En el siguiente ejemplo, el procedimiento `vernombbre` está diseñado para tomar un argumento `Name` de tipo `String` por valor desde un procedimiento de llamada.

```
public void vernombbre(string Name){  
    MessageBox.Show(Name);  
}
```

Por referencia

Se le pasa la dirección de memoria a la que apunta la variable argumento de forma que el procedimiento o función pueden manipular directamente el valor de esa variable. De esta forma, un procedimiento puede modificar las variables enviadas como argumentos.

Un parámetro por referencia se crea con el uso de las palabras reservadas **ref** o **out**.

La etiqueta **ref** se debe utilizar tanto en la lista de parámetros de la función o procedimiento, como en la propia llamada a la función o procedimiento y además debe ser inicializada.

Por el contrario, la etiqueta **“out”** funciona de igual manera pero sin la exigencia de inicializar la variable

Ejemplo:

Declaración de procedimientos con parámetros:

```
//*****Parametros por valor*****
```

```
private void ProcEjemplo1(int variable) {  
    //Instrucciones a ejecutarse  
}  
  
//*****Parametros por referencia*****  
private void ProcEjemplo2(ref int variable)  
{  
    //Instrucciones a ejecutarse  
}  
private void ProcEjemplo2(out int variable)  
{  
    //Instrucciones a ejecutarse  
    variable = 1;  
}
```

Llamada de los procedimientos anteriores

```
private void Form1_Load(object sender, EventArgs e)  
{  
    int x = 0;  
    //parámetros por valor  
    ProcEjemplo1(x);  
  
    //parámetros por referencia  
    ProcEjemplo2(ref x);  
    ProcEjemplo3(out x);  
}
```

Parámetros opcionales

Se puede indicar que un parámetro es opcional asignándole un valor, pero con la precaución de asignar también a todos los parámetros restantes un valor ya que al declarar un parámetro como opcional, el resto de parámetros se vuelven opcionales también.

Ejemplos:

```
double calculoNeto(double Pbruto, double Tasa = 21)
```

```
double calculoNeto(double Pbruto, double Tasa = 21, String divisa="€")
```

La siguiente declaración estaría mal definida ya que el último parámetro ya no es opcional.

```
double calculoNeto(double Pbruto, double Tasa = 21, String divisa);
```

TRANSFIRIENDO ARREGLOS COMO ARGUMENTOS DE SUBROUTINAS

Nos hemos dado cuenta que podemos transferir datos contenidos en variable entre procedimientos y funciones; pero también podemos enviar y recibir Arreglos esto lo hacemos de la siguiente manera:

```
private void ProcesoVoid(params int[] mivector)
```

Como verán al crear el procedimiento (también en función). Para realizar un llamado y enviar un vector o arreglo no necesita indicar el índice

```
private void Form1_Load(object sender, EventArgs e)  
{  
    int[] datos= new int[2];  
    datos[0] = 1;
```

```
    datos[1] = 2;
    ProcesoVoid(datos);
}
```

La palabra clave **params** permite a una función aceptar un número variable de argumentos.

Un parámetro **params** debe declararse como un tipo de matriz unidimensional.

Utilice la palabra clave **params** para denotar una matriz de parámetros. Se aplica las siguientes reglas:

Un procedimiento sólo puede tener una matriz de parámetros, que debe ser el último parámetro de la definición del procedimiento.

- ❖ La matriz de parámetros debe pasarse por valor. Es un hábito de programación recomendado incluir de manera explícita la palabra clave `int`, `string`, `long`, etc.. en la definición del procedimiento.
- ❖ El código del procedimiento debe considerar a la matriz de parámetros una matriz unidimensional; el tipo de datos de los elementos de la matriz ha de ser el mismo que el tipo de datos de **params**.
- ❖ La matriz de parámetros es opcional de forma automática. Su valor predeterminado es una matriz unidimensional vacía del tipo de elemento de la matriz de parámetros.
- ❖ Todos los argumentos que preceden a la matriz de parámetros deben ser obligatorios. La matriz de parámetros debe ser el único argumento opcional.
- ❖ Cuando uno de los argumentos del procedimiento al que se llame sea una matriz de parámetros, ésta podrá tomar cualquiera de estos valores:

Ninguno, es decir, puede omitirse el argumento **params**. En este caso, se pasará una matriz vacía al procedimiento

Una lista con un número de argumentos indeterminado, separados por comas. El tipo de los datos de cada argumento debe poder convertirse implícitamente al tipo de elemento **params**.

Ejemplo de una función con un arreglo de parámetros indeterminados

```
public double media(params double[] notas) {
    double suma=0;
    foreach (double nota in notas) {
        suma = suma + nota;
    }
    return suma /notas.Length;
}
```

Luego, ejemplos de invocación de la función `media()` anterior:

```
double Resultado;
Resultado = media(1,4,67,23.3, 9.8);
Resultado = media(23,69);
```


III. MATERIALES Y EQUIPO

Para la realización de la guía de práctica se requerirá lo siguiente:

No.	Requerimiento	Cantidad
1	Guía de Laboratorio #;Error! No se encuentra el origen de la referencia.	1
2	PC con Microsoft Visual Studio 2012 .NET y con Framework 4.5	1
3	Memoria USB	1

IV. PROCEDIMIENTO

Prepare un nuevo proyecto de Visual Studio 2012.net y guárdelo en la ubicación de su preferencia, para luego asignarle el nombre de **Practica04deLP1**

EJERCICIO 1:

Elabore un programa que permita generar la tabla de multiplicar y de potencias para un numero N dado por usuario. El numero N solo puede variar entre 2.0 y 6.9

* Nota: No puede usar operador matematico potencia (^) ni funciones de math

1. Seleccione la propiedad (Name) del formulario actual y cambie valor por frmGuia04ejerc1
2. En este primer formulario, utilice los controles mostrados en el diseño de pantalla de la figura 1.1. Tenga cuidado de colocar primero al GroupBox y luego ubicar sobre este a los 2 listbox, con el fin que estos ultimos se integren al control contenedor.
3. Luego alterar las propiedades de los controles según valores de la tabla 1.1.

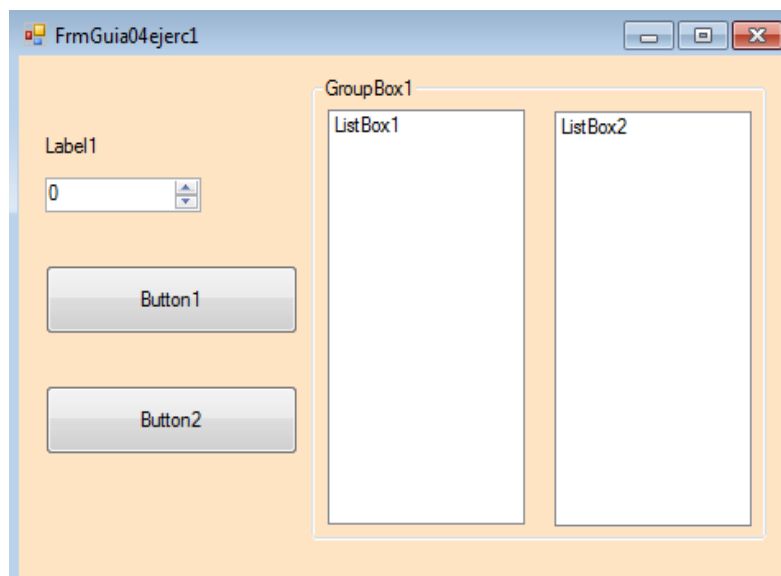


Figura 1.1: Diseño de 1er formulario: controles a utilizar

Propiedades						
(Control)	(name)	Text	Maximum	Minimum	Increment	DecimalPlaces
Label1	lbltema	Ingrese número				
Button1	btnCalculo	Ver tablas				
Button2	btnSalir	Finalizar				
NumericUpDown1	nupBase		6.9	2	0.1	1
GroupBox1	grbResult	Resultados				
ListBox1	lstTabla1					
ListBox2	lstTabla2					

Tabla 1.1: lista de objetos y propiedades a modificar de Form Ejercicio 1

4. Ingrese al editor de código, y seleccione la ubicación (Partial class) de la clase (frmGuia04ejerc0). Luego digite las siguientes subrutinas:

```
//creacion de subrutina a utilizar

private void presentacInic() {
    //prepara controles antes de mostrar form al usuario
    grbResult.Visible = false;
    nupBase.Focus();
}

private void HacerCalculos(Decimal N) {
    /*Se prepara a calcular tablas (de multiplicar y potencia)
    segun valor de variable argumentos N*/
    int c;
    decimal res;

    lstTabla1.Items.Clear();
    c = 1;
    do
    {
        res = N * c;
        lstTabla1.Items.Add(N.ToString() + "X" + c.ToString()+"="+res.ToString());
        c += 1;
    } while (! (c > 10));

    //Genera la portencia de las tablas
    lstTabla2.Items.Clear();
    c = 1;

    do{
        res = Elevar(N, c);
        lstTabla2.Items.Add(N.ToString() + " a la " + c.ToString() + "=" + res.ToString());
        c += 1;
    }while(c<= 10);

}

private decimal Elevar(decimal B, int expo) {
    //reemplaza a operador pow que calcula la potencia de argumentos (Bpow(expo))
    int i = 1;
    decimal r = 1;
    do{
        r *= B;
    }
}
```

```

        i += 1;
    }while(!(i>expo));
    return n;
}

```

Objeto	Evento
<i>btnCalculo</i>	<i>Click</i>
<pre> //Invoca a subrutina HacerCalculos decimal n = nupBase.Value; HacerCalculos(n); //Muestra resultados grbResult.Visible = true; </pre>	
<i>btnSalir</i>	<i>Click</i>
• Instrucción para finalizar aplicación. •	
(Form1 eventos)	Load
• Llamar a función presentacInic.	

- Ahora ejecute el software, escriba un número (entre 2.0 a 6.9), directamente o utilizando las flechas del control. Presione el botón de cálculos y se mostrara el objeto contenedor con ambas listas
- Escriba un nuevo valor para N y presione otra vez el botón de cálculos.
- Analice los resultados obtenidos y presione el 2do boton del form, para terminar ejecución de este ejemplo.

EJERCICIO 2:

Elabore un programa que permita registrar los diferentes montos (\$) de ventas realizadas durante el año 2010, para luego determinar los Montos de ventas (\$) generados por cada trimestre de ese año

Notas:

- Se utilizara 2 controles nuevos: DataGridView y MaskedTextBox
- Desde la ventana del Explorador de soluciones, de clic secundario en el nombre de la solucion y agregue un nuevo Formulario. Guárdelo bajo el nombre **frmGuia04ejerc2.vb**
- Ingresa a las propiedades del proyecto e seleccione a este nuevo form (frmGuia04ejerc2) como Formulario de inicio.
- Guarde los cambios y ejecute el programa, confirme que arranca con el nuevo form!!
- Retorne al modo Diseño y utilice los controles indicados en la figura 1.2 para el diseño de pantalla a implementar en este segundo ejemplo.

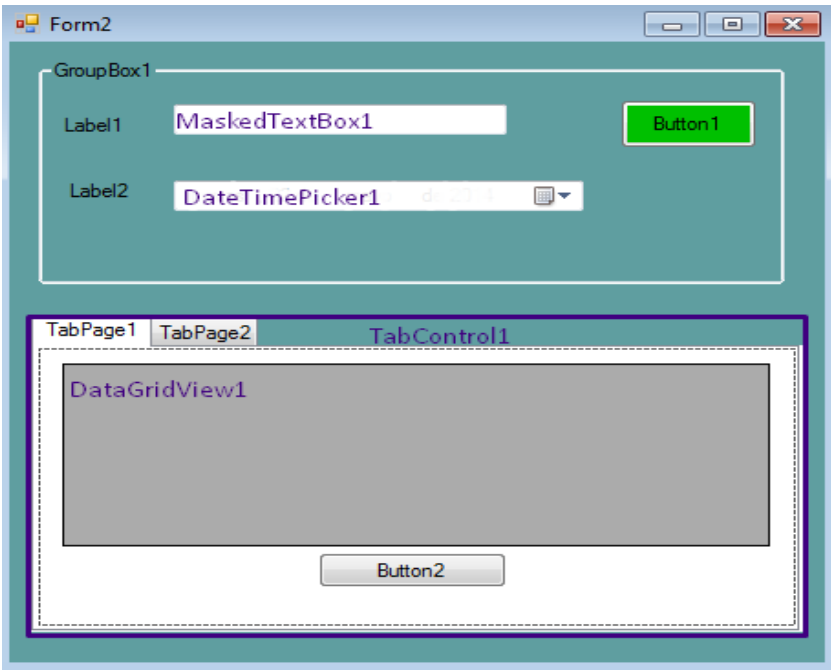


Figura 1.2: Diseño de 2do formulario: controles a utilizar

- 8. Al igual que en el diseño del Form del ejemplo anterior, tenga cuidado en lo siguiente:
 - a) Colocar primero a controles contenedores (GroupBox1 y TabControl1)
 - b) Ubicar controles DataGridView1 y Button2 en la Página TabPage1 del contenedor TabControl1 y el resto en área del otro contenedor GroupBox1
 - c) (IMPORTANTE, no se muestra en imagen): faltan 2 controles más (una Label3 y un Listbox1), los cuales deben agregarse en la Página TabPage2 del contenedor TabControl1
- 9. Por esta vez, no se harán cambios a los nombres de los objetos, con el fin de familiarizarse con los nuevos controles y sus objetos/propiedades/métodos, especialmente el DataGridView
- 10. Modifique las propiedades de los controles definidos en tabla 1.2

Propiedades		
(Control)	Text	Mask
Label1	Monto Venta (\$)	
Label2	Fecha Venta	
Button1	Registrar	
Groupbox1	Detalle de Venta	
Button2	Ver Resumen Ventas	
MaskedTextBox1		00000.00

Tabla 1.2: lista de objetos y propiedades a modificar de Form Ejercicio 1

- 11. Redactar el código siguiente a nivel de la clase **frmGuia04ejerc2.vb**

```
//definicion de procedimientos utilizados
int contaventas;//contador de montos ventas ingresados

public void inicializarControles() {
```

```

/*prepara entorno de trabajo inicial al cargo Form
Mostrara a la pagina 1 del TabControl1 */
tabControl1.TabPages[0].Text= "Ventas efectuadas";
tabControl1.TabPages[1].Text= "Estadísticas de venta";
tabControl1.SelectedIndex = 0;
//configura a DataGridView
dataGridView1.ReadOnly = true;
//Desde coleccion columns, agrega 3 columnas
dataGridView1.Columns.Add("numeventa", "#");
dataGridView1.Columns.Add("montoventa", "Monto ($)");
dataGridView1.Columns.Add("fechaventa", "Fecha Venta");
dataGridView1.Columns.Add("Trime", "Trimestral(1-4)");
/*configura a DateTimePicker
Limita rango de fechas a escoger por usuario*/
dateTimePicker1.MaxDate = Convert.ToDateTime("31/12/2015");
dateTimePicker1.MinDate = Convert.ToDateTime("01/01/2014");
//establece fechas a mostrar (dentro del rango anterior)
dateTimePicker1.Value = Convert.ToDateTime("01/01/2015");
label3.Text = "Totales ventas promedio por trimestre";
maskedTextBox1.Focus();
}

public void registrarVenta(decimal MontoVe, DateTime Fecha)
{
    /*Agrega fila con datos de la venta realizada,
    así como lo clasifica en un trimestre (1-4) del año*/
    int Trimestre;
    //determina el trimestre de parámetros fecha recibido
    switch (Convert.ToInt32(Fecha.Month))
    {
        case 1:
        case 2:
        case 3:
            Trimestre = 1;
            break;
        case 4:
        case 5:
        case 6:
            Trimestre = 2;
            break;
        case 7:
        case 8:
        case 9:
            Trimestre = 3;
            break;
        default:
            Trimestre = 4;
            break;
    }
    dataGridView1.Rows.Add();
    dataGridView1.Rows[contaventas].Cells[0].Value = contaventas + 1;
    dataGridView1.Rows[contaventas].Cells[1].Value = MontoVe;
    dataGridView1.Rows[contaventas].Cells[2].Value = Fecha;
    dataGridView1.Rows[contaventas].Cells[3].Value = Trimestre;
    contaventas++;
}

public void EvaluacionTrimestral() {
    /*Analiza los datos en el grid, para así determinar:

```

```
a)Total ($) de ventas por trimestrs
b)Fecga de la mayor y menor venta efectuada
*/
    decimal[] TotVentaTrim = new decimal[5];
    int c;
    int tri;
    //Primero determina total venta ($) por trimestre
    for (c = 0; c<=(contaventas-1) ; c++)
    {
        /*Recorre c/fila del frid, para comparar el #Trimestre
        de c/venta registrada*/
        tri = Convert.ToInt32( dataGridView1.Rows[c].Cells["Trime"].Value);
        /*Este numero de trimestre se aprovecha para acceder a posicion
        dentro del arreglo TotVentaTrim() aqui acumulo Monto venta de la celda
"montoventa" */
        decimal x =
Convert.ToDecimal(dataGridView1.Rows[c].Cells["montoventa"].Value);
        TotVentaTrim[tri] = TotVentaTrim[tri] + x;
    }
    for (c = 1; c < 5; c++ ) {
        listBox1.Items.Add("Trimestre" + Convert.ToString(c) + ":$" +
Convert.ToString(TotVentaTrim[c]));
    }
}

public void ValidarDatos() {
    //confirmar que usuario escribio datos correctos
    decimal montov;
    //Registra venta en la fecha indicada por usuario
    montov = Convert.ToDecimal(maskedTextBox1.Text);
    //invoca a subrutina, enviando parámetros por valor
    registrarVenta(montov, dateTimePicker1.Value);
    //reinicia controles para nueva venta
    maskedTextBox1.Clear();
    maskedTextBox1.Focus();
}
```

Redacte el código en el siguiente evento del formulario

Objeto	Evento
<i>Button1</i>	<i>Click</i>
• Llamar a función ValidarDatos.	
<i>Button2</i>	<i>Click</i>
• Llamar a función EvaluacionTrimestral. • Copiar siguiente código: //Mostrara a la pagina2 del tabControl1 tabControl1.SelectedIndex =1; //Bloquea ingreso a controles del contenedor Groupbox groupBox1.Enabled = false;	
<i>btnSalir</i>	<i>Click</i>
• Instrucción para finalizar aplicación.	

<i>maskedTextBox1</i>	<i>KeyPress</i>
• Digitar el siguiente código: <pre>//Valida que solo se ingrese numero en el maskedTextBox if (Char.IsNumber(e.KeyChar)) { e.Handled = false; } else { MessageBox.Show("Monto de venta incorrecto"); maskedTextBox1.Focus(); } if (Char.IsLetter(e.KeyChar)) { e.Handled = true; }</pre>	
(Form1 eventos)	Load
• Llamar a función inicializarControles. • Inicializar variable contaventas a cero.	

12. Ahora este código en el evento Clic del botón Button1

13. Y finalmente, este bloque de instrucciones en el evento Clic del Button2

14. Proceda a ejecutar la aplicación. Ingrese unos 5 o más montos de ventas, para luego presionar el botón para ver estadísticas (Montos \$ / trimestre)

EJERCICIO 3:

Uso de procedimientos void, con o sin Parámetros

Elaborar una aplicación Windows en la cual se calculen las 4 operaciones matemáticas básicas (suma, resta, multiplicación, división) y la potencia.

Importante:

Observe los siguientes aspectos en la solución

- ❖ Código se documenta ya sea línea x línea o por bloques, con explicaciones técnicas comprensibles para otros programadores
- ❖ Se redactan primero los procedimientos controlados por evento y luego se agregan los que usted cree para esta aplicación
- ❖ Se utilizan procedimientos tipo subrutinas (**void**) y funciones (return), con o sin parámetros.
- ❖ Algunos argumentos son enviados/pasados por valor u otros por referencia (**ref**)
- ❖ Observe el diseño de c/procedimiento a continuación, los cuales demuestran las diferentes combinaciones a crear con los elementos anteriores.

1. Proceda a crear el siguiente diseño de pantalla en el formulario Form1. Luego asigne las propiedades indicadas en la tabla.

Control	Propiedad(Name)
ComboBox1	cmbOperaciones
Label4	lblResul
NumericUpDown1	nudN1

1. Redactar el código siguiente a nivel de la clase **frmGuia03ejerc3.vb**

```

private void InicializarControles() {
    /*Prepara controles antes de mostrar form a usuario
    Define limites de c/control NumeriUpDown
    (rango valores que aceptara: de -20.0 hasta 35.0)
    */
    nudN1.Minimum = -20;
    nudN1.Maximum = 35;
    nudN1.DecimalPlaces = 1;
    nudN1.Increment = 2;

    nudN2.Minimum = -20;
    nudN2.Maximum = 35;
    nudN2.DecimalPlaces = 1;
    nudN2.Increment = 2;
    //Define presentacion de control cmbOperaciones
    cmbOperaciones.Items.Add("1. Suma");
    cmbOperaciones.Items.Add("2. Resta");
    cmbOperaciones.Items.Add("3. Multiplicación");
    cmbOperaciones.Items.Add("4. División");
    cmbOperaciones.Items.Add("5. Potencia");
    //Listado será solo selección (solo lectura) de valores del combobox
    cmbOperaciones.DropDownStyle = ComboBoxStyle.DropDownList;
    lblResul.Text = "(RESULTADO)";
}

private void SumarEstosNumeros()
{
    //calcula la suma de parámetros A y B recibidos
    double su;
    su = Convert.ToDouble(nudN1.Value) + Convert.ToDouble(nudN2.Value);
    /*Muestra resultado de suma (operac 1) y como no hay
    error no se envia ultimo parámetro*/
    MostrarResultado(su, 1);
}

private void MostrarResultado(double R, int Oper, Boolean hayError = false)
{

```



```

        /*Muestre respuesta en label, resalta en
        A. fondo verde y letra blanca (si es error)
        B. fondo rojo y letra amarilla (si hay errores)
        copia texto del item seleccionado en cmbOperaciones*/
        string descripOpe = cmbOperaciones.Text;
        if (!hayError)
        {
            lblResul.BackColor = Color.Green;
            lblResul.ForeColor = Color.White;
            lblResul.Text = descripOpe + "es " + Convert.ToString(R);
        }
        else
        {
            lblResul.BackColor = Color.Red;
            lblResul.ForeColor = Color.Yellow;
            lblResul.Text = "(ERROR, DIVISION POR CERO)";
        }
    }

    private double Potencia(double A, double B)
    {
        /*Calcula potencia de A^B
        Retorna potencia a llamada recursiva anterior de misma función potencia*/
        return Math.Pow(A, B);
    }

    private void Multiplicar(double x, double y, ref double M)
    {
        /*Recibe 2 factores X e Y de parámetros entrada, para luego
        retornar en parámetro de salida (M) al resultado de multiplicación*/
        M = x * y;
    }

    private Boolean Dividir(double x, double y, ref double d)
    {
        /*Recibe 2 parámetros (x e y) de entrada, para intentar dividirlos y guardar resultados en 0
        confirma si hay division entre cero(0)*/
        if (y == 0.0)
        {
            //retorna False porque la division noo puede hacerse
            return (false);
        }
        else
        {
            /*Hace division y retorna True indicando que operacion se realizo*/
            d = x / y;
            return true;
        }
    }

    private void RestarA(double a, double b)
    {
        /*Calcula la suma de parámetros A y B recibidos*/
        double su;
        int opc;
        //Toma valores escritos en controles NumericUpDown
        su = a - b;
        opc = cmbOperaciones.SelectedIndex + 1;
        /*Muestra resultado de suma (operac 1) y como
        no hay error no se envia ultimo parámetro*/
        MostrarResultado(su, 1);
    }

```

```
private void HacerOperacion(int numOperac) {
switch (numOperac){
    case 1:
        SumarEstosNumeros();
        break;
    case 2:
        RestarA(Convert.ToDouble(nudN1.Value), Convert.ToDouble(nudN2.Value));
        break;
    case 3:
        double prod = 0;
        Multiplicar(Convert.ToDouble(nudN1.Value),Convert.ToDouble(nudN2.Value),ref prod);
        MostrarResultado(prod, 3, false);
        break;
    case 4:
        double division=0;
        if (Dividir(Convert.ToDouble(nudN1.Value),Convert.ToDouble(nudN2.Value),ref division)){
            MostrarResultado(division, 3);
        }
        else{
            MostrarResultado(division,4,true);
        }
        break;
    case 5:
        MostrarResultado(Potencia(Convert.ToDouble(nudN1.Value),Convert.ToDouble(nudN2.Value)),5);
        break;
    default:
        MessageBox.Show("Operación solicitada no valida", "ERROR", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
        break;
}
}
```

2. Redacte el código en el siguiente evento del formulario

Objeto	Evento
<i>cmbOperaciones</i>	<i>SelectedIndexChanged</i>
• Digitar el siguiente código: <pre>/*Hace operaciones matematicas especifica cuando usuario selecciona item del combo*/ int nop; /*num de operacion selecciona en comobo toma indice(0-4) seleccionado actualmente del cmboperaciones, para obtener num operacion solicitada*/ nop = cmbOperaciones.SelectedIndex + 1; //procede a realizar operacion matematica indicada HacerOperacion(nop);</pre>	
(Form1 eventos)	Load
• Llamar a función inicializarControles.	

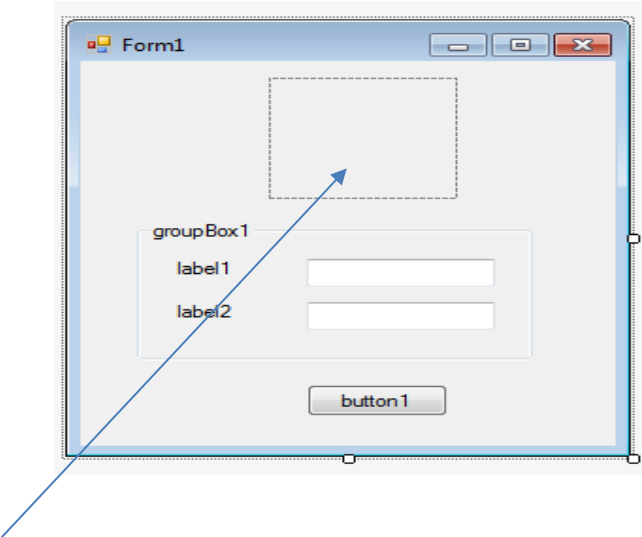
EJERCICIO 4:

Uso de procedimientos return, con o sin Parámetros

En este ejercicio se trabajará con tres formularios, de los cuales dos están desarrollados en esta guía práctica y el tercero lo hará ud. tomando de referencia algunos puntos tocados en este formulario.

1. Proceda a crear el siguiente diseño de pantalla en el formulario Form1. Luego asigne las propiedades indicadas en la tabla.

Figura1. Diseño de Formulario (Form1)



No	Elemento	Name	Text
1	Form		Login
2	groupBox1		Private
3	Label1		Usuario:
4	Label2		Password:
5	Button1	btnAceptar	Aceptar
6	Textbox1	txtusuario	
7	Textbox2	txtpwd	

NOTA: Para poder insertar la imagen que irá en el control, debe indicarlo en la propiedad Image, al dar clic aparecerá una venta igual a la que se muestra en la figura2.

Elemento	Name	Image
PictureBox	pictureBox1	Imagen seleccionada

Figura2. Se selecciona la imagen.

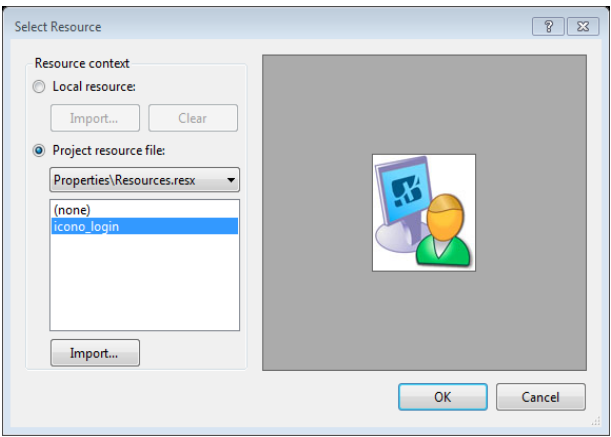
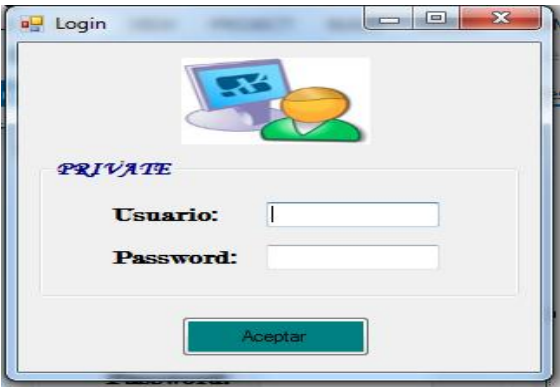


Figura3. Formulario ejecutándose



NOTA: La imagen contenida en el control PictureBox, queda a criterio de cada estudiante, así como el color de fondo del control y el formato de estilo del texto.

3. Declare a nivel de la clase Form4, la siguiente función:

```
private Boolean validar(string nombre, string pwd)
{
    string clave = nombre;
    string pasword = "usuario";
    DialogResult respuesta; // variable para capturar el dato que me devuelve el
    MessageBox.Show

    if (nombre == clave && pwd == pasword)
    {

        respuesta = MessageBox.Show("Bienvenido" + " " + nombre, "Acceso",
        MessageBoxButtons.OK, MessageBoxIcon.Asterisk);

        if (respuesta == DialogResult.OK)
        {
            /* Para llamar a otro formulario(Form5),se debe primero instancia al
            nuevo formulario,o es decir creamos el objeto
            * para nuestro caso formulario2 y luego accesamos al método show, para
            mostrar el Form5*/
            Form5 formulario2 = new Form5();//instanciando al Form
            formulario2.Show();    // Mostramos el Form2
            return true;
        }
    }//Fin de if
    else {
        MessageBox.Show("Contraseña
        incorrecta","Acceso",MessageBoxButtons.OK,MessageBoxIcon.Error);
    }
    return false;
}// Fin de Función
```

4. Redacte el código en el siguiente evento del formulario

Objeto	Evento
<i>Form1</i>	<i>Load</i>
• Digitar el siguiente código: <pre>txtusuario.Focus();</pre>	
<i>btnAceptar</i>	<i>Click</i>
<pre>if(validar(txtusuario.Text, txtpwd.Text)){ // Se llama la función validar declarada. this.Hide(); }// ocultamos el Form4</pre>	

ASIGNACIÓN: Agregue el código necesario para incorporar las validaciones que se necesitan, para asegurar que la información que ingrese el usuario sea válida, por ejemplo: que las dos casillas donde se pide información no estén vacías, en caso de introducir contraseña incorrecta, enviar el focus al control, para que el usuario digite la clave correcta y todas las que crea necesarias.

5. Agregar un nuevo formulario al proyecto (Este formulario será invocado por el primero, para simular los montos a cancelar (facturación) por hospedaje en un lugar x).

NOTA: Antes de poner los controles Checkbox, primero debe poner el control contenedor y luego dentro del contenedor debe poner los checkbox.

Control: NumericUpDown

Figura 4: Diseño de formulario (Form5)

No	Elemento	Name	Text	DecimalPlaces
1	label1	label1	Apellidos y Nombres	
2	label2	label2	Días de hospedaje	
3	Textbox1	txtnombre		
4	Textbox2	txtdias		
5	groupBox1	groupBox1	Tipo	
6	groupBox2	groupBox2	Servicios	
7	radioButton1	radioButton1	Turista	
8	radioButton2	radioButton2	Delegado	
9	checkBox1	checkBox1	Cable	
10	checkBox2	checkBox2	Teléfono/Fax	
11	checkBox3	checkBox3	Bar libre	
12	button1	button1	Calcular	
13	button2	button2	Nuevo	
14	button3	button3	Acerca...	
15	button4	button4	Salir	
16	label3	label3	Monto de Hospedaje:	
17	label4	label4	Monto de Servicio	
18	label5	label5	Monto Total:	

No	Elemento	Name	Text	DecimalPlaces
19	Textbox3	txtmontoh		
20	Textbox4	txtmontos		
21	Textbox5	txtmontot		
22	Textbox6	txtinteres		
23	NumericUpDown	nUDpagar		2

6. Declare a nivel de la clase Form2, la siguiente funciones, contantes y variable:

```
const float Pago_turista = 50, Pago_Delegado = 70;
float interes = 0.18f;

private float calculo_hospedaje(int dias) // Función recibe un parámetro y retorna un valor.
{
    float calculo = 0f;
    if (radioButton1.Checked)
    {
        calculo = Convert.ToInt32(txtdias.Text) * Pago_turista;
    }
    else
    {
        calculo = Convert.ToInt32(txtdias.Text) * Pago_Delegado;
    }

    return calculo;
}

private int calculo_servicio()// Función no recibe parámetros pero devuelve un valor
{
    int acum = 0;

    /* Usar CheckBox sin tanto if. Para ello se recorre la colección ControlCollection de la
    propiedad Controls,
    después se verifica si el control es un CheckBox y de serlo, verificar su valor.
    Cuando damos click en el boton calcular se invoca esta función y se recorren todos los controles
    del formulario, y si un control es un CheckBox, entonces se verifica su valor y si es verdadero
    acumulamos el valor del servicio*/

    foreach (Control contr in this.groupBox2.Controls)
    {
        CheckBox checkbox = contr as CheckBox;
        if (checkbox.Checked)
            acum += 20;
    } // fin de primer if dentro de foreach
    return acum;
}
```

7. Digite el código detallado a continuación, en los eventos y objetos especificados.

Objeto	Evento
From2	Load
<pre> txtnombre.Focus(); txtmontoh.Enabled = false; txtmontos.Enabled = false; txtmontot.Enabled = false; txtinteres.Enabled = false; nUDpagar.Enabled = false; radioButton1.Checked = true; checkBox1.Checked = false; checkBox2.Checked = false; checkBox3.Checked = false; </pre>	
button1	Click
<pre> float montoh = 0f; int montos = 0; decimal total = 0; montoh= calculo_hospedaje(Convert.ToInt32 (txtdias.Text)); txtmontoh.Text = Convert.ToString(montoh); montos=calculo_servicio(); txtmontos.Text = Convert.ToString(montos); txtmontot.Text= Convert.ToString(montoh+montos); txtinteres.Text = ((float.Parse(txtmontot.Text) * interes)).ToString(); total= Convert.ToDecimal(txtmontot.Text) + Convert.ToDecimal(txtinteres.Text); nUDpagar.Maximum= 2*total; nUDpagar.Value = total; </pre>	
button2	Click
<pre> txtnombre.Text = ""; txtnombre.Focus(); txtdias.Text = " "; txtmontoh.Text = ""; txtmontos.Text = ""; txtmontot.Text = ""; txtinteres.Text = ""; nUDpagar.Value =0; radioButton1.Checked = true; checkBox1.Checked = false; checkBox2.Checked = false; checkBox3.Checked = false; </pre>	

Button4	Click
<code>this.Close();</code>	

ASIGNACIÓN:

Agregue el código necesario para incorporar las validaciones que se necesitan, para asegurar que la información que ingrese el usuario sea válida, por ejemplo:

Que las dos primeros Textbox donde se pide información, por ejemplo el Textbox donde se pide el número de días, solo debe aceptar números enteros, debe seleccionar al menos un control de checkBox y todas las que crea necesarias.

V. DISCUSION DE RESULTADOS

PROBLEMAS A RESOLVER:

Para los siguientes ejercicios, realizar un solo proyecto de Aplicación Windows, con la resolución de los problemas en forms distintos:

1. Cree una serie de metodos, que reciban como parámetro a un vector unidimensional para simular el funcionamiento de una **Lista**. Los métodos a implementar seran:
 - a) **Crear lista vacia:** Crea al vector recibido en un parametro con cero elementos.
 - b) **Mostrar listado elementos de la lista:** Recibe como parametros al arreglo que simula la Lista y un control ListBox. Este retornara en ListBox recibido a los elementos existentes del listado.
 - c) **Insertar elemento en lista:** Recibe como parametros al arreglo que simula la lista y un valor a agregar. El nuevo valor debe ser agregado al final del listado recibido.
 - d) **Remove elemento:** Elimina el primer elemento del vector recibido como parametro.

Importante:

* El arreglo que simula el listado de elementos solo puede transferirse en forma de parametros.

* Para modificar el tamaño del vector, investigue a la clase Array.

* La demostración de cada método solicitado, se hará clic sobre un botón diferente. Si es necesario, en un botón pueden invocarse varios de estos métodos, para demostrar el funcionamiento de uno de ellos.

2. Redactar un método que retorne a una matriz de 5X5, llena con números enteros aleatorios, cada uno de los cuales estarán ubicados entre 2 numeros limites recibidos como parametros.
Cree otro método que reciba la matriz cuadrada generada previamente y la muestre en un DataGridView.

3. Escribir un método que reciba una lista de N notas numéricas (cada una solamente entre cero a 10.0) y un control de ListBox.

El método mostrara en el cuadro ListBox a los siguientes resultados estadísticos:

- a) Porcentaje de estudiantes Deficientes (con notas menores de 5.0)
- b) Número de aprobados (con notas mayores o iguales a 6.0)
- c) Identificar la nota más baja y la más alta del listado, así como la nota media de todo el grupo.

VI. BIBLIOGRAFÍA

- Título: Aprenda ya Microsoft Visual C#.NET.
Autores: Sharp, John Jagger, Jon Coautor
Publisher: Madrid, España: McGraw-Hill, 2002.
Clasificación: Libro 005.362 S581 2002
- Título: C # Manual de Programación.
Autores: Luis Joyanes Aguilar y Matilde Fernández Azuela
Publisher: Madrid, España: McGraw-Hill, 2002
Clasificación: Libro 005.362 J88 2002